

课程笔记

通向通用智能体：Qwen 2025 技术进展

五道口纳什 & Codex

2026 年 3 月 28 日



视频作者/频道：AITIME 论道

发布日期：2025 年

视频时长：约 38 分钟

视频链接：未填写

目录

1 引言：从 Model 到 Agent	2
1.1 本章小结	2
2 Qwen3 文本模型：Reasoning 的自然化	2
2.1 千问三系列的核心升级	2
2.2 Reasoning 的自然化之路	3
2.3 多语言：从韩语到乌尔都语	3
2.4 长上下文与 Memory	4
2.5 本章小结	4
3 Coding Agent：从竞赛题到真实工程	4
3.1 Coder 系列的进化	4
3.2 训练范式的变革：Agent RL	5
3.3 本章小结	5
4 视觉语言模型 (VL)：给 AI 装上眼睛	5
4.1 核心目标：Human Level Visual Understanding	5
4.2 四大提升方向	5
4.3 本章小结	6
5 图像生成与编辑：从理解到创造	6
5.1 Create Image：接近真实的生成质量	6
5.2 图像编辑：比生成更刚需	6
5.3 本章小结	7
6 语音模型 (Omni)：听与说的统一	7
6.1 Thinker-Talker 架构	7
6.2 本章小结	7
7 模型架构：Qwen3-Next 与 Linear Attention	7
7.1 混合注意力架构	7
7.2 本章小结	8
8 未来路线：Generalist Agent	8
8.1 三进三出的全模态模型	8
8.2 Multi-turn RL with Environment Feedback	8
8.3 从 Digital Agent 到 Physical Agent	8
8.4 本章小结	8
9 总结与延伸	9
9.1 拓展阅读	9

1 引言：从 Model 到 Agent

林俊堃 (Qwen 团队负责人) 在清华-北大联合实验室的活动中, 介绍了通义千问 (Qwen) 2025 年的技术进展。演讲主题从最初的 “Towards a Generalist Model” 改为 “Towards a Generalist Agent” ——这一改变反映了整个 AI 领域的范式转移：从 “训练更强的模型” 到 “构建能自主使用工具的智能体”。

为什么从 Model 到 Agent ?

林俊堃认为, Agent 是一个比 Model 更大的概念。人类与动物的核心差异之一在于**自主使用工具的能力**。而今天的训练范式变化 (特别是强化学习的引入) 使得 AI 也获得了这种可能: 只要解决了推理和评估, 模型就能被训练去执行各种任务——无论是 Digital Agent 还是 Physical Agent。

Qwen 开源生态概览

- 2023 年 8 月 3 日开始开源之路
- 体验入口: chat.qwen.ai (聚合闭源和开源模型)
- 模型发布平台: Hugging Face 和 ModelScope (内容同步)
- GitHub 以脚本为主; 技术博客代替论文 (如 Qwen3-Next 架构等)
- 吉祥物是一只水豚 (Capybara), 取名佛系风格

1.1 本章小结

Qwen 团队的愿景是构建一个 “Multimodal Foundation Agent”——一个能看 (视觉)、能听 (语音)、能说 (生成)、能做 (工具使用) 的通用智能体。这一愿景从 2020 年的多模态研究延续至今。

2 Qwen3 文本模型：Reasoning 的自然化

2.1 千问三系列的核心升级

相比被广泛使用的 Qwen 2.5, 千问三的主要提升包括:

1. **整体能力提升**: 作为新一代基座模型的全面进化
2. **Reasoning 能力跃升**: 这是 2024–2025 年最显著的变化
3. **多语言支持**: 支持 119 种语言及方言
4. **长上下文**: 已做到 1M+ token, 内部实验已达数 M 级别

2.2 Reasoning 的自然化之路

Thinking 与 Instruct 的统一

林俊堃分享了一个重要的市场信号：在 4 月份发布 Thinking+Instruct 合并版本后，**超过 90% 的客户放弃了单独的 Thinking 模型**，回到了 Instruct 模型。原因是：

- 用户最初喜欢看模型的“自我对话”（thinking trace），但这种新鲜感很快消退
- Instruct 模型更实用，响应更自然
- 关键是如何让 Reasoning 能力**自然地融入**到 Instruct 模型中

Qwen 团队在 7 月份推出了改进版本，核心做法是：对于 Reasoning-intensive 的任务，将 Thinking 过程集成到回答本身中，而非使用独立的 thinking token。效果是 AIME 等数学基准从约 20 分提升到约 70 分（Thinking 模型可达 90 分），客户反馈模型“明显聪明了很多”。

DeepSeek R1 的启示与反思

DeepSeek R1 展示了通过“絮絮叨叨的自我对话”可以让模型变强，但 Qwen 团队认为这不是最终形态。他们追求的是**更自然的 Reasoning**——模型在需要深度思考时确实会思考，但不会用冗长的内心独白打扰用户体验。

2.3 多语言：从韩语到乌尔都语

多语言支持的起源是一个有趣的故事：

1. 最初 Qwen 只做了中英双语
2. 一位韩国研究者说“你们的模型完全不懂韩语”，选择了 Mistral 作为基座
3. 调查发现 Pretrain 模型其实会韩语，只是 Post-train 没有覆盖
4. 顺手修复后，全球用户开始涌入
5. 巴基斯坦用户反复请求支持乌尔都语——“我们真的没有大模型可用”
6. 目前已支持 119 种语言，但非洲语言覆盖仍不足

数字鸿沟的现实

林俊堃提到，非洲很多地区至今仍在使用的功能机。如果做大模型的想法不是帮助全人类，“那还不如不做”。这一理想主义立场驱动了 Qwen 持续扩展语言覆盖的努力。

2.4 长上下文与 Memory

Context Length 与 RAG 的关系

林俊堃引用了 Gemini 长序列团队 leader 的观点：Context Length 和 RAG 本质上是正交的。

- 不应该把大量“垃圾”塞入长上下文
- 更长的上下文为更好的记忆管理提供了基础
- Qwen 已做到 1M+ token，内部实验达数 M 级别
- 改变模型架构（Qwen3-Next）的动机之一就是探索 infinite long context 的可能

2.5 本章小结

Qwen3 文本模型的核心进步在于将 Reasoning 能力自然地融入 Instruct 模型中，同时扩展多语言支持和长上下文能力。“让模型聪明但不烦人”是产品化的关键平衡。

3 Coding Agent：从竞赛题到真实工程

3.1 Coder 系列的进化

林俊堃指出，今天的 Coding 模型与过去有本质不同：

- **过去**：解竞赛题（给题目，输出 Solution）——对应 Live Code Bench
- **今天**：Software Engineering（看 GitHub issue，提 PR 修复）——对应 SWE-Bench

Agent 式编程的本质

维护一个 GitHub 仓库的流程看似简单，但实际涉及多轮环境交互：打开文件夹、阅读文件名、理解代码结构、定位问题、修改代码、运行测试……这正是 Agent 的核心——**与环境的多轮交互**。

Qwen Coder 在 SWE-Bench 上达到 67–69 分（7 月版本），Terminal-Bench 等 CLI 任务基准上也表现出色。更重要的是，这些 Coding 能力已被集成到最大的通用模型（大于 1T 参数）中，SWE-Bench 同样可达约 69 分。

中美 Coding 文化差异

林俊堃从湾区回来后感受到一个有趣的差异：Cognition (Devin) 的创始人说“旧金山已经没人手写代码了，所有人都在 vibe coding”。但在中国，vibe coding 的普及度还不够高。不过殊途同归——大家的目标都是让 AI 真正产生生产力。

3.2 训练范式的变革：Agent RL

Agent 训练的新范式

Qwen 的 Coding Agent 训练不再是传统的输入-输出对标注，而是：

1. 设定 Schedule（训练任务队列）
2. 将 Agent（LM + Scaffold）部署到 Docker 环境中
3. Agent 在环境中自主操作（打开文件、编辑代码、运行测试等）
4. 根据结果给予 reward 信号
5. 销毁环境，进入下一轮

这是真正意义上的 Multi-turn RL with Environment Feedback，不仅是算法挑战，还涉及大量 Infra 工程（Docker 管理、并行训练、状态同步等）。

算法-Infra 联合优化

过去“算法 Infra 联合优化”常被视为噱头，但在 Agent 训练时代，这是**真实存在的需求**。除了 DeepSeek 提到的训练效率提升外，Agent 训练需要大量的环境管理 Infra 支持——这已经超出了纯算法团队的能力范畴。

3.3 本章小结

Coding Agent 代表了从“解题”到“工程”的范式转变。Qwen 通过 Agent RL 在真实环境中训练模型，并将 Coding 能力整合到统一的大模型中。未来的评测也应从静态 Benchmark 转向动态的真实世界任务。

4 视觉语言模型（VL）：给 AI 装上眼睛

4.1 核心目标：Human Level Visual Understanding

Qwen-VL 的目标是让模型拥有“人类水平的视觉理解”——尽管在某些方面（如细节识别），模型已经超越人类，但在基础的空间关系（上下左右）判断上仍有不足。

VL 模型的“降质”问题

长期以来，将视觉能力加入语言模型时会导致语言智力下降（称为“降质”）。Qwen-VL 最新版本首次实现了**语言理解不降质**——VL 模型的文本能力与 235B 纯文本模型基本持平。

4.2 四大提升方向

1. **GUI/手机操控**：提升操控电脑和手机的能力——Agent 需要能“看到”并“操作”屏幕
2. **语言智力保持**：VL 模型可以直接当 LM 使用，追赶 Gemini 系列的原生多模态能力
3. **多模态 Coding**：代码的输入可以是图像或视频（画个草图就能生成 App），降低 prompt 编写门槛

4. **视觉 Reasoning**: 结合 Thinking 能力进行视觉推理——例如数一张照片中有多少人（通过逐个标注来计数）

第一人称视角与 VLA

林俊^[6]提到一个前沿探索方向：如果有了智能眼镜，每天接收的第一人称视角（Egocentric Video）数据能否用来构建 memory？这驱动了 Qwen 向 VLA（Vision-Language-Action）方向的探索——将机器人控制数据和第一人称视角数据融入模型训练。不过 VLA 目前连基本的 Scaling Law 都未探通。

4.3 本章小结

Qwen-VL 在保持语言智力不降的前提下，大幅提升了视觉理解能力。下一步是让 VL 模型同时具备视觉 Reasoning 和 GUI 操控能力，并探索 VLA 方向。

5 图像生成与编辑：从理解到创造

5.1 Create Image：接近真实的生成质量

Qwen 于 2025 年开始图像生成方向，Create Image 系列在几个月内取得了显著进展：

- 8 月版本：AI 感很重
- 12 月版本：接近照片级真实感（近乎“离谱”的逼真度）
- 内部 Arena 排名：超越最佳开源模型，接近 GPT-4.5 和 Gemini 3 Pro
- 特色能力：一次性生成带文字的分镜图（12 格漫画 + 对话文字，非拼接而是单张生成）

5.2 图像编辑：比生成更刚需

开源社区的意外发现

Qwen 团队发现，图像**编辑**可能是比生成更大的需求。一个典型案例：用户想“放下图中人物的右手”，编辑后将两张图叠加时发现人物位置发生了偏移（shift）。这对 PS 专业用户来说是不可接受的精度问题。

2511 版本专门解决了这一问题——编辑后人物保持在原位，blending 后几乎无偏差。这个 case 来自开源社区的反馈，“如果不是开源社区告诉我们，可能这辈子都想不到有这个问题”。

其他编辑能力包括：光线调整、物体移除、风格迁移、镜头旋转等。用户对图片物理合理性（如光影关系）的要求远超算法工程师的直觉。

理解-生成一体化的价值

一个令人兴奋的应用场景是**辅助线绘制**：在教育领域教小朋友做几何题时，AI 光靠文字无法展示辅助线的画法——这需要图像生成能力与数学推理的结合。这是理解-生成一体化模型独特的价值所在。

5.3 本章小结

图像生成和编辑是 Qwen “全模态” 愿景的重要拼图。开源社区的反馈在真实场景需求发现中起到了不可替代的作用。下一步是将编辑与生成统一到一个模型中。

6 语音模型 (Omni)：听与说的统一

6.1 Thinker-Talker 架构

Qwen-Omni 采用了一套 “Thinker + Talker” 架构：

- **Thinker**：负责理解和推理
- **Talker**：负责语音合成和输出
- 实现了 end-to-end 的语音交互（不依赖 ASR 管线）

端到端语音 vs ASR 管线

传统做法是 ASR → LLM → TTS 三段式管线，但 Qwen-Omni 走 end-to-end 路线——模型直接理解语音输入并生成语音输出。这不仅减少了延迟，还能捕捉语调、情感等非文字信息。TTS 方面可通过 prompt 描述声音特征来定制音色。

当前 Omni 模型的语言智力略有降质（约 Gemini 2.5 Flash 水平），但语音能力对标 Gemini 2.5 Pro。

6.2 本章小结

语音是 Agent 与人类交互的重要通道。Qwen-Omni 的 Thinker-Talker 架构是实现 “三进三出”（文本、视觉、音频的理解与生成）目标的关键一步。

7 模型架构：Qwen3-Next 与 Linear Attention

7.1 混合注意力架构

Qwen3-Next 采用了 **3:1 Hybrid 架构**——三层 Linear Attention 配一层 Full Attention 交替堆叠。这一设计与月之暗面（Kimi）的架构 “殊途同归”。

为什么要换架构？

核心动机有三：

1. **降低 KV Cache 成本**：Linear Attention 不需要维护完整的 KV Cache，大幅节省显存
2. **支持超长上下文**：为 infinite long context 铺路
3. **推理效率**：长序列场景下推理速度更快

此外，Qwen 团队的 Attention Gate 机制获得了 NeurIPS Best Paper。

7.2 本章小结

架构创新是支撑下一代能力(超长上下文、Agent 持续交互)的基础设施。Qwen3-Next 的 Hybrid Linear Attention 架构在保持模型质量的同时,为长程 Agent 交互提供了必要的效率保障。

8 未来路线: Generalist Agent

8.1 三进三出的全模态模型

Qwen 的终极目标是一个能够同时**理解**和**生成**文本、视觉、音频三种模态的统一模型:

- 已实现: 文本理解/生成、视觉理解、音频理解/生成
- 待实现: 将视觉生成融入统一模型
- 终极形态: “三进三出”——三种模态自由输入输出

8.2 Multi-turn RL with Environment Feedback

Long Horizon Reasoning

林俊堃认为 RL 的未来不是做 DeepSeek R1 式的数学竞赛自对话,而是 **Multi-turn RL with Environment Feedback for Long Horizon Reasoning**:

- 模型在真实环境中执行长链任务(如花两天完成人类两个月的工作)
- 通过环境反馈进行学习,而非仅靠自我对话
- 适用于虚拟世界(Digital Agent)和物理世界(Embodied Agent)
- 本质上是将 Natural Language Instruction 转换为 Executable Action

RL Compute 的趋势

XAI 的宣传显示其 RL 计算量已接近 Pre-training 计算量。虽然林俊堃认为“有点浪费”,但这确实说明 RL 有巨大的想象空间。Qwen 团队更关心的不是让模型成为最强数学大脑,而是让它能像真实的人一样为生物和社会做贡献。

8.3 从 Digital Agent 到 Physical Agent

- **Digital Agent**: 同时操作 GUI 和 API,是当前最成熟的方向
- **Physical Agent**: 操控话筒、倒水等物理世界任务,需要 VLA (Vision-Language-Action) 支持
- 统一视角: 无论 Coding Agent 还是 VLA,本质都是“将自然语言指令转化为可执行动作”

8.4 本章小结

Qwen 的技术路线清晰: 全模态统一 → Agent RL → Digital/Physical Agent。这是一条从“理解世界”到“改变世界”的进化之路。

9 总结与延伸

林俊堃这次演讲展现了 Qwen 团队在 2025 年的全景图，可以从三个维度总结：

1. 模型能力的全面进化

- 文本：Reasoning 自然化、119 种语言、超长上下文
- 视觉：VL 不降质、GUI 操控、视觉 Reasoning
- 生成：图像生成接近真实、编辑精度大幅提升
- 语音：Thinker-Talker 端到端架构
- 架构：Qwen3-Next Hybrid Linear Attention

2. 从 Model 到 Agent 的范式转变

- Coding Agent 通过环境交互式 RL 训练
- SWE-Bench 67–69 分，能力集成到超大统一模型
- 目标从解题转向真实世界的软件工程任务

3. 开源生态的正向循环

- 小模型（1.8B）源于“让硕士生能毕业”的初心
- 多语言源于韩国研究者和巴基斯坦用户的反馈
- 图像编辑精度源于开源社区发现的偏移问题
- “如果不是开源社区告诉我们，可能这辈子都想不到”

9.1 拓展阅读

- Qwen 官方博客：<https://qwen.ai>
- Qwen Chat 体验：<https://chat.qwen.ai>
- Qwen3-Next 架构博客（含 Attention Gate NeurIPS Best Paper）
- SWE-Bench 基准：<https://swebench.com>
- OpenHands（原 OpenDevin）：开源 Agent 框架